Universal AegisValidator

User guide — install, operate, interpret

Version 1.0.0 | single Python file | standard library only
Validator sha256 `735c7849d169933a…`

An offline-first, authorized-use validator for AI guardrail behaviour. It runs a versioned corpus of defensive tests against a model endpoint (or a deterministic local simulator), classifies every execution as PASS / FAIL / INCONCLUSIVE / ERROR, and writes a machine-readable JSON report plus a multi-page colour PDF. It never collapses results into a single “safety score”, and it never pretends a simulated run is a live one.

Quick facts

240 built-in tests · 21 corpus tests in 14 categories · 8 provider profiles + your own · 4 outcome states · 0 third-party dependencies · JSON + PDF on every run

In this guide

Quickstart · reading the report · live testing safely · provider profiles · CI gate and exit codes · SIEM, alerts and cloud storage · tool runner · credential refresh · web UI · troubleshooting · limitations

IMPORTANT Live mode may only be used against AI systems you own or have explicit written authorization to test. The tool refuses unapproved targets by design.

Contents

1. What this tool does (and does not do) . . . 3

2. Requirements and installation . . . 3

3. Quickstart in five commands . . . 3

4. Reading the results . . . 4

4.1 The four outcome states . . . 4

4.2 Statistics that refuse to over-claim . . . 4

4.3 Anatomy of the PDF . . . 4

5. Live testing, safely . . . 4

6. Provider profiles . . . 5

7. The corpus, mutations and reproducibility . . . 6

8. CI/CD gate and exit codes . . . 6

9. SIEM logging and alerting . . . 7

9.1 Audit log formats . . . 7

9.2 Alert routing . . . 7

10. Shipping reports out of ephemeral containers . . . 8

11. Tool-call sandbox and the hardened runner . . . 8

12. Long runs: short-lived credentials . . . 8

13. The development web UI . . . 9

14. Troubleshooting . . . 9

15. Security posture and honest limitations . . . 9

16. Command reference . . . 10

1. What this tool does (and does not do)

AegisValidator performs **black-box behavioural testing** of a model endpoint's guardrails. It sends a curated corpus of adversarial-but-safe prompts, mutates them in meaning-preserving ways, and compares what came back against expectations the operator declared in the corpus.

It does	It does not
Send documented defensive tests and record the raw evidence	Exploit systems, steal credentials, persist, or evade detection
Classify each execution into one of four explicit states	Produce a single opaque “safety score” or a CVSS rating
Mutate prompts in ways that preserve intent, to test robustness	Claim that a finite corpus proves a model is safe
Refuse live targets that were not explicitly authorized	Guess authorization from the URL or from a config file
Write PDF + JSON evidence for every completed run	Silently omit a report when findings are inconvenient

NOTE Severity in this tool expresses **configured impact**, not exploitability. A CRITICAL finding means “the corpus said this matters and the endpoint did not meet the expectation” — not “this model is exploitable”.

2. Requirements and installation

There is nothing to install. The validator is a single file with no third-party dependencies; the PDF writer, the HTTP transport, the Cisco/Splunk log encoders and the cloud request signers are all implemented against the standard library.

Requirement	Detail
Python	3.9 or newer (grammar-verified for 3.9; exercised on 3.13)
Packages	none — no pip install, no virtualenv, no lockfile
Network	optional; simulate mode performs zero network I/O
Disk	writes a PDF (~60 KB) and JSON (~300 KB) per run
Credentials	only for live mode, supplied through environment variables

```
# verify the download and see what it can do
python3 universal_aegis_validator.py --version
python3 universal_aegis_validator.py --list-profiles
python3 universal_aegis_validator.py --list-categories

# run its own test suite (240 tests, fully offline)
python3 universal_aegis_validator.py --run-tests
```

3. Quickstart in five commands

```
# 1. offline end-to-end audit: no network, no credentials, deterministic
python3 universal_aegis_validator.py --mode simulate

# 2. same, but smaller and faster
python3 universal_aegis_validator.py --mode simulate --max-tests 6 --variants-per-test 2

# 3. reproducible run (same seed -> same result, byte for byte)
python3 universal_aegis_validator.py --mode simulate --seed 1234 --report-dir ./reports

# 4. render the local web UI instead of the terminal (loopback only)
python3 universal_aegis_validator.py --web

# 5. live assessment of a target you are authorized to test
export AEGIS_OPENAI_API_KEY='...'
python3 universal_aegis_validator.py --mode live --profile openai_chat \
    --authorize-target https://api.openai.com --max-tests 20 --budget 60
```

Every completed audit prints the absolute paths of both artefacts:

```
REPORTS WRITTEN
  PDF : /work/reports/aegis_audit_20261029_130639_...-28a559.pdf
  JSON: /work/reports/aegis_audit_20261029_130639_...-28a559.json
  summary: 74 execution(s): PASS 50 FAIL 7 INCONCLUSIVE 12 ERROR 5
```

4. Reading the results

4.1 The four outcome states

State	Meaning	What to do
`PASS`	The response met every expectation the corpus declared for that test.	Nothing; it is evidence of a control working, for this corpus only.
`FAIL`	A declared expectation was violated (forbidden marker, canary echo, unauthorized tool, missing required schema).	Read the evidence card, reproduce with the printed command, fix or re-test.
`INCONCLUSIVE`	The response exists but cannot be judged automatically (ambiguous, truncated, no applicable rule).	Route to human review; the tool refuses to guess.
`ERROR`	No usable response: timeout, connection failure, non-2xx, unparseable body.	Fix infrastructure; ERROR is never counted as a behavioural failure.

NOTE Numbers you can quote: **behavioural pass rate** = PASS / (PASS + FAIL) — ERROR and INCONCLUSIVE are excluded, because infrastructure noise is not behaviour. **Execution reliability** = valid executions / planned executions, reported separately.

4.2 Statistics that refuse to over-claim

A 95% Wilson interval is attached to the pass rate only when the sample supports it (n ≥ 5). Below that the interval is withheld and the JSON says why (`behavioral_pass_rate_interval_withheld_reason`), because a 1-of-1 run has no meaningful confidence interval.

Behavioural consistency between a baseline and its mutated variant uses `min(token Jaccard, character 3-gram Jaccard)`. Token similarity alone is blind to rewrites that keep every word but change the bytes a tokenizer sees (hyphenation, zero-width padding); the character measure catches those.

4.3 Anatomy of the PDF

Page	Contents
`1`	Cover: target, profile, mode (LIVE / SIMULATED), run id, verdict summary
`2`	Dashboard: outcome counts, pass rate with interval, severity mix, latency percentiles
`3`	Findings: one row per failing or reviewable vector, ordered by severity
`4`	Methodology: coverage, formulas, execution conditions, known limitations
`5+`	Evidence cards: sanitised input, sanitised response, expected vs observed, reproduction command, digest
`last`	Production operations posture: gate, storage, audit-log format, alert format, tool-execution posture, prompt composition, credential source

5. Live testing, safely

Live mode is guarded by four independent controls. All four must pass before the first request leaves the process.

Control	Behaviour
Explicit authorization	`--authorize-target` must name the exact host you are authorized to test; a mismatch refuses the run (exit 3).
Credential presence	The profile's auth environment variable must be set; a missing credential refuses the run instead of sending an anonymous request.
Network policy	Loopback, private ranges and cloud metadata addresses are refused unless `--allow-private-network` (and `--allow-insecure-http` for plain http) is given.
Request budget	`--budget` caps total requests; redirects are rejected, response size is capped, retries are bounded.

```
# authorized SAS/OpenAI-compatible target
export AEGIS_OPENAI_API_KEY='sk-...'
python3 universal_aegis_validator.py --mode live --profile openai_chat \
    --authorize-target https://api.openai.com --max-tests 20 --variants-per-test 2 --budget 60

# local inference server (plain http + private range need explicit opt-in)
export AEGIS_VLLM_API_KEY='local'
python3 universal_aegis_validator.py --mode live --profile vllm_openai \
    --base-url http://127.0.0.1:8000 --authorize-target http://127.0.0.1:8000 \
    --allow-private-network --allow-insecure-http --budget 30
```

WARNING Live results describe **one endpoint, one model, one corpus, one moment**. Re-run after model, prompt, or gateway changes; treat a single green run as a snapshot, not a certification.

6. Provider profiles

A profile is a configurable template — base URL, endpoint, auth header, payload skeleton, prompt path, response path. No profile ships with a credential, and no profile is marked “live-validated” until an operator runs it against their own endpoint.

```
python3 universal_aegis_validator.py --list-profiles
#   openai_chat           OpenAI-compatible Chat Completions
#   compatible_chat       vendors that mirror the OpenAI request shape
#   anthropic_messages    Anthropic Messages API
#   google_gemini         Gemini generateContent
#   ollama_local          local inference (needs --allow-private-network)
#   vllm_openai           self-hosted vLLM deployment
#   localai_openai        any other OpenAI-compatible local server
```

Add your own without touching the file — a JSON array of profile objects. This one also turns on streaming, which the extractor stitches back into a normal completion:

```
[{
  "provider": "vllm", "profile_id": "vllm_streaming",
  "display_name": "vLLM streaming deployment",
  "base_url": "http://127.0.0.1:8099", "endpoint_path": "/v1/chat/completions",
  "method": "POST",
  "headers": {"Content-Type": "application/json", "Accept": "text/event-stream"},
  "auth_env_var": "AEGIS_VLLM_API_KEY",
  "auth_header": "Authorization", "auth_prefix": "Bearer ",
  "payload_template": {"model": "{{MODEL}}",
                       "messages": [{"role": "user", "content": "{{PROMPT}}"}],
                       "stream": true, "temperature": 0},
  "prompt_path": "messages[0].content",
  "response_path": "choices[0].message.content",
  "model": "my-model", "approved_hosts": ["127.0.0.1"], "approved_ports": [8099],
  "allow_insecure_http": true
}]

python3 universal_aegis_validator.py --mode live --profile vllm_streaming \
    --profile-file ./my_profiles.json --authorize-target http://127.0.0.1:8099 \
    --allow-private-network --allow-insecure-http
```

Profiles whose payload carries a native role array (`messages` or Gemini `contents`) receive **structured messages**: system context, conversation history and the mutated user turn are appended as separate objects. Templates without such an array fall back to a single composed string. The report records which strategy was used.

7. The corpus, mutations and reproducibility

Flag	Effect
`--categories`	Comma-separated subset, e.g. prompt_injection,tool_use_authorization
`--max-tests`	Limit how many test cases run (0 = all)
`--intensity`	Mutation intensity 1..3 (default 2)
`--max-chain`	Maximum mutation chain length 1..4
`--variants-per-test`	Mutated variants per test case (0..12)
`--seed`	Deterministic seed; same seed + same corpus = same result
`--corpus-file`	Import extra tests from JSONL
`--export-corpus`	Write the built-in corpus to JSONL and exit
`--baseline-report`	Compare against an earlier JSON report (regression mode)

```
# export the corpus, extend it, feed it back
python3 universal_aegis_validator.py --export-corpus corpus.jsonl
python3 universal_aegis_validator.py --corpus-file corpus.jsonl --mode simulate

# regression: did this release make anything worse?
python3 universal_aegis_validator.py --mode live --profile openai_chat \
    --authorize-target https://api.openai.com --baseline-report ./previous.json
```

Baselines are captured in a first sequential pass and then **frozen** before the concurrent pass starts, so a worker can never compare against a baseline that changed mid-run. The report records a digest of the frozen reference set.

8. CI/CD gate and exit codes

```
python3 universal_aegis_validator.py --mode live --profile openai_chat \
    --authorize-target https://api.openai.com --max-tests 21 --variants-per-test 2 --budget 60 \
    --gate-policy ci/gate-policy.json
```

Exit	Meaning
`0`	Audit complete and every gate rule passed
`1`	Gate failed on a behavioural invariant (pass rate, severity ceiling, weight)
`2`	Configuration error (bad profile, policy, storage, alert or runner config)
`3`	Refused for safety/authorization (target not approved, credential missing)
`4`	Report generation or storage failure — no usable artefacts
`5`	Gate failed on an infrastructure invariant (ERROR ceiling, min_executions, partial run)
`130`	Interrupted (Ctrl-C); whatever completed is still written

A gate policy is a small JSON document. Every rule is evaluated — never short-circuited — and the per-rule verdict is written to `<report>.gate.json`:

```
{
  "policy_id": "prod-guardrail-v1",
  "execution_mode": "live",
  "allow_simulated": false,
  "min_executions": 40,
  "min_behavioral_pass_rate": 0.95,
  "max_failures_critical": 0,
  "max_failures_high": 2,
  "max_error_rate": 0.05,
  "max_total_severity_weight": 25,
  "required_categories": ["prompt_injection", "tool_use_authorization"],
  "run_complete": true
}
```

NOTE Reports are written **before** the gate verdict, so a failing pipeline still has its PDF, JSON and gate manifest as evidence. A SIMULATED run can never satisfy a production gate unless `allow_simulated` is explicitly set.

9. SIEM logging and alerting

9.1 Audit log formats

```
--audit-log ./reports/audit.jsonl                          # JSON Lines (default)
--audit-log ./reports/audit.cef --audit-log-format cef      # ArcSight / QRadar / Sentinel
--audit-log ./reports/audit.leef --audit-log-format leef    # QRadar LEEF
--audit-log ./reports/audit.ecs --audit-log-format ecs      # Elastic Common Schema 8.11.0
--audit-log-max-bytes 10485760                              # rotate to <name>.1
--log-json                                                  # app logs as JSON on stderr
```

Every record carries a monotonic `seq` plus `event`, `run_id`, `service` and `version`. All free-text fields pass through the same credential redactor as the reports.

9.2 Alert routing

```
export SECOPS_SLACK_WEBHOOK='https://hooks.slack.com/services/T.../B.../xxx'
export AEGIS_SPLUNK_HEC_TOKEN='...'                      # splunk format only

python3 universal_aegis_validator.py --mode live ... \
    --alert-allow-host hooks.slack.com \                # mandatory allow-list
    --alert-min-severity high --alert-max 3 \
    --alert-format slack        # default
    --alert-format teams        # MessageCard payload
    --alert-format splunk       # HEC envelope + Authorization: Splunk <token>
    --alert-format generic      # {"text": ...} for anything Slack-compatible
--alert-dry-run               # build and print the payload, send nothing
```

Only findings at or above the minimum severity (plus a failing gate) trigger a message; the payload is redacted and bounded. A delivery failure is reported in the run summary and the audit log, but never fails an audit and never flips a gate.

10. Shipping reports out of ephemeral containers

Scheme	Use	Credentials
`dir:`	Mounted volume (Kubernetes PVC, EFS)	none
`presign:`	Pre-signed URL — signature lives in the URL	none (URL carries it)
`s3://`	S3 or S3-compatible (MinIO, LocalStack)	AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY (+ AWS_SESSION_TOKEN)
`gs://`	Google Cloud Storage	AEGIS_GCS_TOKEN or GOOGLE_OAUTH_ACCESS_TOKEN
`az://`	Azure Blob Storage	AZURE_STORAGE_ACCOUNT / AZURE_STORAGE_KEY

```
        --storage-uri "dir:/mnt/aegis-reports"
        --storage-uri "s3://bucket/prefix" --storage-region eu-west-1
        --storage-uri "s3://bucket/prefix" --storage-endpoint http://minio.internal:9000
        --storage-allow-host 127.0.0.1 --storage-allow-insecure-http     # loopback MinIO
        --storage-prune-local                                           # delete after success only
```

Requests are signed in-process: SigV4 for S3, an OAuth bearer token for GCS, SharedKey HMAC for Azure. Credentials stay in memory, are never written to the report, the audit log or the alert payload, and an upload failure leaves the local files untouched.

11. Tool-call sandbox and the hardened runner

`--tool-sandbox` parses model-emitted tool calls and classifies each attempt: **denied** (destructive deny-list), **unknown** (unregistered = unauthorized), **allowed** (operator allow-list). Nothing executes unless you also supply a runner *and* the explicit master switch.

```
        python3 universal_aegis_validator.py --mode live ... --tool-sandbox \
            --tool-runner-command "docker run --rm -i --network=none --read-only \
                --memory=128m --pids-limit=64 my/aegis-tools:1" \
            --tool-runner-allow-execute        # master switch
            --tool-runner-timeout 5 --tool-runner-max-output 4096 \
            --tool-runner-max-executions 20
```

Guarantee	How it is enforced
No shell	argv list, shell=False; sh/bash/cmd/powershell templates are rejected
Model cannot craft a command line	Tool arguments are delivered on stdin as JSON
No credential leakage	Child environment rebuilt from PATH/LANG/LC_ALL/HOME only
Isolation required	Template must contain --network=none and --read-only, or exit 2
Bounded	Per-call timeout, output cap, per-run execution budget, cwd forced to a temp dir
Auditable	exit code, stdout, stderr, timeout flag, duration are journaled per vector
Never scored	Execution success/failure is evidence, not a PASS/FAIL outcome

12. Long runs: short-lived credentials

OIDC and STS tokens expire. Instead of pasting a static value, nominate a command that mints a fresh one; it is run on demand, without a shell, with a scrubbed environment:

```
python3 universal_aegis_validator.py --mode live --profile openai_chat \
    --authorize-target https://api.openai.com \
    --credential-refresh-command "oidc-agent --account secops get-access-token" \
    --credential-refresh-interval 50 \      # re-mint every N requests (0 = only on expiry)
    --credential-expiry-skew 30 \           # refresh this many seconds before expiry
    --credential-refresh-timeout 10
```

stdout may be `{"token": "...", "expires_in": 3600}` or a bare token. The token is cached in memory only; the report records that refreshes happened and when, never the value. A failed refresh becomes an `authentication_failed` error for that vector instead of crashing the run.

13. The development web UI

```
python3 universal_aegis_validator.py --web                # http://127.0.0.1:8765
python3 universal_aegis_validator.py --web --web-port 9000

# DEVELOPMENT ONLY - single-user, unauthenticated, never expose to an untrusted network
--web-host 0.0.0.0 --allow-non-loopback-web
```

The UI offers the same corpus and reporting as the CLI: start a scan, watch progress, and download the PDF when it finishes. It binds to loopback by default and refuses a non-loopback bind without the explicit development flag.

14. Troubleshooting

Symptom	Likely cause	Fix
exit 2, “profile host does not match”	--authorize-target names a different host than the profile	Use --base-url or --authorize-target so both point at the same host
exit 3, “requires --authorize-target”	Live mode without an authorization string	Add the exact host you are authorized to test
exit 3, “credential environment variable is not set”	Profile auth env var missing	export it, or use --credential-refresh-command
All vectors ERROR, connection refused	Target down or wrong port	Check --base-url; for loopback add --allow-private-network --allow-insecure-http
FAIL on the first baseline of a category	The endpoint's normal behaviour already violates a declared expectation	Read the evidence card; the corpus expectation may need to match your product policy
“sample too small for a 95% interval”	Fewer than 5 PASS+FAIL vectors	Run more tests; the interval is deliberately withheld rather than faked
Gate exit 1 with execution_mode [simulate]	A production policy rejects simulated runs	Run live, or use a policy with allow_simulated for pipeline plumbing tests
Tool runner exits 2, missing isolation flags	Template lacks --network=none / --read-only	Add them (recommended) or --tool-runner-unsafe
Streaming target returns empty text	Body is SSE and not stitched	Confirm the profile sends stream:true; the extractor stitches SSE and NDJSON automatically
PDF characters shown as '?'	Base-14 fonts cover cp1252 only	Expected and disclosed in the report; counts are printed in the methodology page

15. Security posture and honest limitations

- No third-party code path: the transport, PDF writer, signers and log encoders are standard library only, so there is no dependency supply chain to audit.

- Evidence is sanitised before it is stored or rendered: API keys, bearer tokens, JWTs, AWS key IDs, SAS parameters and pre-signed signatures are replaced with [REDACTED_*] markers.
- Redirects are rejected, response bodies are size-capped, DNS/SSRF checks run before every request, and the request budget bounds the blast radius of a misconfiguration.
- Tool execution is isolated by *your* container runtime, not by this file: the validator enforces argv-only invocation, a scrubbed environment, timeouts, output caps and a budget, but the containment boundary is the runtime you configure.
- A finite corpus cannot prove the absence of vulnerabilities; the report says so on every page set that quotes a pass rate.
- Simulated runs never demonstrate how a real model behaves; the report labels every affect vector SIMULATED and a production gate rejects simulated evidence by default.
- Severity weights express configured impact, not exploitability, and no single score is ever produced.

IMPORTANT Use live mode only against AI systems you own or have explicit written authorization to test. The tool contains no exploitation, credential theft, stealth, persistence or destructive capability.

16. Command reference

Generated from the shipped argument parser, so it matches this build exactly.

```
CORE
  --mode {simulate,live}          execution mode (default: simulate)
  --profile ID                    provider profile (--list-profiles)
  --profile-file FILE             JSON array of extra profiles
  --model NAME                    override the profile's model id
  --base-url URL                  override the profile's base URL
  --authorize-target URL          REQUIRED in live mode: the host you may test
  --allow-private-network         permit loopback/private destinations
  --allow-insecure-http           permit plain http (local targets only)
  --budget N                      maximum live requests
  --timeout SECONDS               request timeout override
  --seed N                        deterministic seed
  --concurrency N                 bounded workers (1..16)

CORPUS
  --categories a,b,c              category subset
  --max-tests N                   limit test cases (0 = all)
  --intensity 1..3                mutation intensity
  --max-chain 1..4                mutation chain length
  --variants-per-test 0..12       mutated variants per test
  --corpus-file FILE              import extra tests (JSONL)
  --export-corpus FILE            export the built-in corpus and exit
  --baseline-report FILE          regression comparison against a JSON report

REPORTING
  --report-dir DIR                output directory (default: ./aegis_reports)
  --allow-overwrite               permit overwriting existing reports
  --no-evidence                   omit evidence pages
  --evidence-max-chars N          excerpt length in reports
  --retain-raw-evidence           keep untruncated (still redacted) JSON evidence
  --max-evidence-vectors N        vectors rendered on evidence pages
  --open-pdf                      open the PDF when finished

GATE
  --gate                          evaluate the CI gate after the scan
  --gate-policy FILE              JSON policy document
  --min-pass-rate 0.95            minimum behavioural pass rate
  --max-fail-severity c,h         FAIL ceilings ('critical=0,high=2')
  --max-errors N                  infrastructure ERROR ceiling
  --require-categories a,b        categories that must be covered
  --min-executions N              minimum executed vectors
  --allow-simulated-gate          let a SIMULATED run satisfy the gate
```

```
SIEM / ALERTS
  --audit-log FILE                append audit events (JSONL by default)
  --audit-log-format {jsonl,cef,leef,ecs}
  --audit-log-max-bytes N         rotate to <name>.1 past this size
  --log-json                      application logs as JSON on stderr
  --alert-webhook-url URL         alert webhook (prefer --alert-webhook-env)
  --alert-webhook-env NAME        env var holding the webhook
  --alert-allow-host HOST         REQUIRED allow-list for delivery
  --alert-min-severity {critical,high,medium,low,info}
  --alert-max N                   findings per alert
  --alert-format {slack,teams,splunk,generic}
  --alert-splunk-token-env NAME   env var holding the Splunk HEC token
  --alert-dry-run                 build, print, send nothing

STORAGE
  --storage-uri dir:/path | presign:https://.../{filename} | s3://b/p | gs://b/p | az://c/p
  --storage-allow-host HOST       REQUIRED allow-list for presign/cloud hosts
  --storage-allow-insecure-http   permit plain-http endpoints (MinIO on loopback)
  --storage-region REGION         AWS region for s3://
  --storage-endpoint URL          S3/Azure-compatible endpoint
  --storage-prune-local           delete local copies after a successful upload

TOOL EXECUTION
  --tool-sandbox                  parse and journal tool calls
  --tool-runner-command "argv"    container runtime command template
  --tool-runner-allow-execute     master switch: actually execute allow-listed tools
  --tool-runner-unsafe            skip the isolation-flag check (weaker posture, recorded)
  --tool-runner-timeout SECONDS   per-call timeout
  --tool-runner-max-output BYTES  captured output cap
  --tool-runner-max-executions N  per-run execution budget

CREDENTIALS
  --credential-refresh-command "argv"   command that mints a fresh token
  --credential-refresh-env NAME         env var the token stands in for
  --credential-refresh-interval N       re-mint after N requests (0 = on expiry)
  --credential-refresh-timeout SECONDS  refresh command timeout
  --credential-expiry-skew SECONDS      refresh early by this much

INTERFACE
  --web [--web-host H] [--web-port P]   local development UI (loopback by default)
  --allow-non-loopback-web              DEVELOPMENT ONLY
  --list-profiles / --list-categories / --run-tests
  --no-color / --quiet / --log-level LEVEL / --version
```

End of guide. Documentation generated from the validator build identified on the cover; re-run this guide's build script after changing the tool so the reference stays exact.

Universal AegisValidator is for authorized testing only. Use live mode exclusively against AI systems you own or have explicit written permission to assess.